Invention Disclosure

Scoped Telepathy for Multi-Agent Shared Memory Systems

Inventor: Paul Finton (AJ)

Date of Invention: April 28, 2026

Prepared for evaluation by patent counsel

This document is a technical disclosure, not a patent application

1. Title

Scoped Event Routing with Self-Exclusion for Multi-Agent Shared Memory (Telepathy)

2. Inventors

Paul Finton (AJ)

3. Date of Invention

April 28, 2026

4. Field of the Invention

This invention relates to real-time event distribution in multi-agent artificial intelligence systems. More specifically, it relates to a method for sharing memory state changes between autonomous AI agents operating on a shared memory store, where each agent receives only the changes relevant to its operational scope and never receives notifications of its own writes.

5. Background and Problem Statement

5.1 The Multi-Agent Problem

Modern AI agent systems (e.g., coding assistants, monitoring agents, personal assistants, DevOps agents) increasingly operate in teams. Multiple agents may be running simultaneously - a development agent writing code, a monitoring agent watching infrastructure, a personal agent managing user preferences - all working on behalf of the same user or organization.

These agents need shared context. When the development agent learns that a server has moved to a new IP address, the monitoring agent should know immediately, not wait for the user to repeat the information.

5.2 Prior Art Approaches

Approach 1: Frozen context.

Each agent loads its context once at startup and never updates it. This is simple but guarantees stale data. In a long-running session, one agent's context diverges from reality as other agents write new memories.

Approach 2: Polling.

Agents periodically re-query the memory store. This creates unnecessary load when nothing has changed, adds latency when something has, and doesn't scale - N agents polling at T intervals generates N/T queries per second.

Approach 3: Broadcast all events to all agents.

A naive pub/sub approach where every write event is sent to every connected agent. This causes information overload - a monitoring agent doesn't need the personal agent's "user prefers dark mode" update, and vice versa.

Approach 4: Agent-to-agent messaging.

Agents explicitly send messages to specific other agents. This requires agents to know about each other, creates tight coupling, and doesn't handle the common case where the writing agent doesn't know who needs its information.

5.3 The Unsolved Problem

No existing system provides a method for:

1. Automatically routing memory change events only to agents whose operational scope overlaps with the

change, AND
2. Preventing an agent from receiving notifications of its own writes, AND
3. Doing this without polling, broadcasting, or explicit agent-to-agent addressing, AND
4. Allowing each agent to define its scope declaratively (by target, category, and key prefix).

6. Summary of the Invention

The invention, called Telepathy, is a scoped event routing system for multi-agent shared memory. It provides real-time awareness between autonomous AI agents operating on a shared memory store, where:

- Agents subscribe to changes matching their operational scope (target, category, key prefix).
- The daemon routes change events only to agents whose subscriptions match the change.
- Self-exclusion prevents an agent from receiving its own write events.
- Hot block invalidation triggers re-computation of an agent's working context when a relevant change arrives.

The result is that agents gain shared awareness - "telepathy" - without polling, broadcasting, or explicit inter-agent communication. Agent B writes a memory; Agent A's working context updates within milliseconds, automatically.

7. Detailed Description

7.1 System Architecture

The system comprises three components:

1. A shared memory daemon (single process) that owns the memory store (SQLite with full-text search) and serves all agents via gRPC.
2. A pub/sub hub (in-process, within the daemon) that maintains subscriptions and routes events.
3. Agent-side client libraries that subscribe to change streams and maintain hot block caches.

All agents connect to the same daemon instance. The daemon is the sole writer to the database, ensuring serialization of all mutations.

7.2 Subscription Model

Each agent declares a subscription that specifies which memory changes it cares about, using three dimensions:

Dimension	Description	Example
Target	Memory namespace: "memory" or "user"	Dev agent subscribes to "memory"
Category	Type: "environment", "project", etc.	Agent boosting "project" subscribes to those
Key Prefix	Prefix match on key field	Agent watching servers uses prefix "server-"

A subscription that omits a dimension matches all values for that dimension, allowing broad or narrow scope.

7.3 Self-Exclusion

When an agent writes a memory, the daemon publishes a change event to the pub/sub hub. The hub's routing logic explicitly checks whether the subscription's agent_id matches the event's source_agent_id. If they match, the event is withheld.

Why this matters:

Without self-exclusion, every write by Agent A would cause Agent A to receive a notification about its own write. This is redundant and creates a feedback loop. Self-exclusion breaks this loop at the routing layer - agents don't need filtering code.

7.4 Hot Block Invalidation

Each agent maintains a hot block - a compact, rendered text block of its most important memories, injected into the

LLM's system prompt. The hot block is computed by the daemon based on per-agent parameters:

- Character limit (e.g., 4000 characters for memory, 2000 for user data)
- Minimum priority threshold (e.g., priority >= 2)
- Category boost list (e.g., boost "project" and "environment" for a dev agent)
- Category suppress list (e.g., suppress "observation" for a dev agent)

When a telepathy event arrives, the agent knows its hot block may be stale. It can request a fresh hot block from the daemon, or the daemon can pre-compute it and include it in the event (HOT_BLOCK_CHANGED event type).

7.5 Event Lifecycle

1. Agent A connects to daemon, calls Subscribe with its scope parameters.
2. Daemon opens a server-side streaming RPC to Agent A.
3. Agent B calls AddMemory to write "server IP is 10.0.0.5" with target="memory", category="environment", key="server-ip".
4. Daemon persists the memory to SQLite.
5. Daemon publishes an event to the pub/sub hub.
6. Hub evaluates all subscriptions: Agent A matches and is not self-excluded. Agent B would match but is self-excluded.
7. Agent A receives the event on its subscription stream.
8. Agent A requests a fresh hot block from the daemon (or the event includes it).
9. Agent A replaces its cached hot block with the fresh one.
10. On Agent A's next LLM call, the system prompt includes the updated hot block - Agent A knows about Agent B's discovery within milliseconds.

7.6 Disconnection Cleanup

When an agent disconnects, the daemon detects stream cancellation via gRPC context and cleans up subscriptions. The hub also provides UnsubscribeAll(agentID) for heartbeat timeout scenarios.

8. Novel Aspects and Distinctions from Prior Art

8.1 Scoped Routing vs. Broadcast Pub/Sub

Existing pub/sub systems (MQTT, NATS, Kafka) allow topic-based routing, but topics are opaque strings. The invention uses structured, semantic scope matching (target x category x key prefix) that maps directly to the agent's hot block computation parameters:

- An agent's subscription is derived directly from its hot block configuration - no separate routing config needed.
- The scope dimensions correspond to the memory schema, not arbitrary topic strings.
- An agent that changes its hot block configuration automatically changes its subscription scope.

8.2 Self-Exclusion at the Routing Layer

Existing pub/sub systems deliver all matching events to all subscribers, including events published by the subscriber itself. The invention performs self-exclusion at the routing layer, before the event is ever placed on the subscriber's channel. This:

- Eliminates the feedback loop at its source.
- Reduces unnecessary network traffic and processing.
- Prevents subtle bugs where an agent reacts to its own write and creates an infinite loop.
- Requires no application-layer filtering code in the agent.

8.3 Hot Block Invalidation via Event Stream

Existing agent systems either freeze context at session start or poll for updates. The invention uses the event stream as an invalidation signal:

- Lower latency than polling: Events arrive within milliseconds.
- Lower cost than polling: No queries when nothing has changed.
- Freshness guarantee: Hot block is re-computed only when known stale.

8.4 Single-Writer Architecture with gRPC Streaming

The daemon is the sole writer to SQLite. All agents read and write through gRPC. This eliminates write contention, ensures all writes trigger event publication, and allows atomic hot block computation.

9. Potential Claims

The following are potential claim areas for evaluation by patent counsel:

Claim 1:

A method for routing memory change events in a multi-agent system, comprising: maintaining a set of subscriptions each defined by an agent identifier, a target scope, a category scope, and a key prefix scope; receiving a write event from a first agent, the write event including a target, a category, a key, and a source agent identifier; evaluating each subscription against the write event by matching the target, category, and key against the subscription's corresponding scopes; withholding the event from any subscription whose agent identifier matches the source agent identifier (self-exclusion); and delivering the event to subscriptions that match and are not self-excluded.

Claim 2:

The method of claim 1, wherein the subscriptions are declarative - an agent specifies its scope without knowledge of other agents, and the routing is performed by a central daemon without inter-agent communication.

Claim 3:

The method of claim 1, wherein the target, category, and key prefix scopes correspond directly to the parameters used to compute the agent's hot block (a compact, rendered summary of the agent's most relevant memories), such that an agent's subscription is derived from its hot block configuration without additional routing logic.

Claim 4:

A method for maintaining contextual awareness between autonomous AI agents, comprising: maintaining a shared memory store accessible to a plurality of agents; computing, for each agent, a hot block comprising a subset of memories selected by priority, target, and category parameters; publishing an event when a memory is written to the shared store; selectively routing the event to agents whose hot block parameters overlap with the written memory, excluding the writing agent; and invalidating the hot block of each receiving agent responsive to the event.

Claim 5:

The method of claim 4, wherein hot block invalidation triggers re-computation of the hot block by a central daemon, and the recomputed hot block is delivered to the agent as part of the event or via a separate request.

Claim 6:

A system for scoped event routing in a multi-agent shared memory environment, comprising: a shared memory daemon configured to persist memories and compute hot blocks; a publish/subscribe hub within the daemon, the hub maintaining subscriptions that specify target, category, and key prefix scopes; self-exclusion logic within the hub that withholds events from the agent that originated the write; and a gRPC server-streaming connection between the daemon and each agent for real-time event delivery.

Claim 7:

The method of claim 1, further comprising automatically cleaning up subscriptions when an agent disconnects, using context cancellation from the streaming connection.

10. Prior Art Search Areas

For the patent attorney's evaluation:

Patent Class	Search Terms
G06N (AI)	multi-agent memory sharing, agent context awareness
G06F 16/00 (Info retrieval)	scoped publish subscribe, semantic event routing
H04L 67/00 (Network protocols)	self-exclusion publish subscribe, agent identity routing
G06F 9/54 (IPC)	shared memory notification, invalidation-based cache update

Known related systems to distinguish from:

- MQTT/NATS/Kafka - topic-based pub/sub, no structured scope matching, no self-exclusion
- Redis Pub/Sub - channel-based, no semantic scope, no self-exclusion
- LangChain/LangGraph - agent orchestration, but agents don't share memory through scoped events
- AutoGen (Microsoft) - multi-agent chat, but agents communicate via explicit messages
- CrewAI - agent delegation, but no shared memory with scoped invalidation
- OpenAI Assistants API - single-agent context, no multi-agent memory sharing
- MemGPT/Letta - single-agent memory management, no multi-agent telepathy

11. Commercial Application

This invention is implemented in the Mnemosyne project (github.com/aj-nt/mnemosyne), an open-source shared memory daemon for AI agents. The telepathy system is the core differentiator that enables multi-agent awareness without polling, broadcasting, or explicit inter-agent messaging.

Potential commercial applications:

- Multi-agent coding teams (one agent writes architecture decisions, another reads them in real-time)
- DevOps automation (monitoring agent detects incident, development agent receives the alert)
- Personal AI assistants (one agent learns user preferences, all others reflect them immediately)
- Enterprise knowledge management (department-specific agents share findings across boundaries)

12. Drawings / Figures

Figure 1: System Architecture

The following diagram shows the daemon, pub/sub hub, and multiple agents connected via gRPC streaming:

```
Agent A                Shared Memory Daemon
  (dev)                   +------------+
                          | SQLite     |
  Subscribe:              | Store      |
  target:"memory"         +------------+
  categories:                   |write
  project,                      |
  environment                   v
  ------->stream           AddMemory(RPC)
       |                        |
       |                        v
       |                  Pub/Sub Hub
       |                  +------------------+
       |                  | Subscription:    |
       |                  |   Agent A:       |
       |                  |   target=memory  |
       |                  |   cat=proj,env   |
       |                  |   Agent B:       |
       |                  |   EXCLUDED       |
       |                  |   (self-write)   |
       |                  +-------+----------+
       |                          |
       |   Event routed to A      |
       |   (matches, not self-    |
```

```
      |    excluded)             |
      <.......................+
      |
      v
   Agent A hot block updates
   to include Agent B's discovery
```

Figure 2: Self-Exclusion Routing Logic

When a write event arrives, the hub evaluates each subscription:

```
Write Event: {target: "memory", category: "environment",
              key: "server-ip", source: "agent-b"}

Agent A subscription:
  agent_id: "agent-a"
  target: "memory"
  category: "project", "environment"
  ---
  Target match?    target="memory"    -> YES
  Category match?  "environment"      -> YES
  Self-exclusion? "agent-a" != "agent-b" -> PASS
  ---
  ROUTE: Agent A receives event

Agent B subscription (if any):
  agent_id: "agent-b"
  target: "memory"
  ---
  Target match?    target="memory"    -> YES
  Category match?  (none = all)      -> YES
  Self-exclusion? "agent-b" = "agent-b" -> BLOCKED
  ---
  WITHHOLD: Agent B does NOT receive its own write
```

Figure 3: Hot Block Invalidation Lifecycle

```
Agent A's hot block contains:
  [environment] server-ip: 10.0.0.1
  [project] mnemosyne: Memory daemon in Go

Agent B writes: "server-ip is now 10.0.0.5"

Event arrives at Agent A -> hot block known stale

Agent A requests fresh hot block from daemon

Agent A's updated hot block:
  [environment] server-ip: 10.0.0.5    <- UPDATED
  [project] mnemosyne: Memory daemon in Go

Agent A's next LLM call includes the updated context
-> Agent A now knows about the server change
-> Without polling, without being told, without asking
-> Telepathy.
```

13. Implementation Reference

The invention is implemented in Go, approximately 160 lines for the pub/sub hub, 50 lines for the subscription routing logic, and 30 lines for the self-exclusion check. The daemon communicates with agents via gRPC server-streaming (protoc-generated code). The full source code is available at github.com/aj-nt/mnemosyne under the MIT license.

Key source files:

- daemon/internal/pubsub/hub.go - Hub, Subscribe, Publish, matches, UnsubscribeAll
- daemon/internal/grpc/service.go - Subscribe RPC, AddMemory (with event publishing)
- client/hotblock_cache.go - Client-side hot block cache with Update method
- proto/mnemosyne.proto - SubscribeRequest, UpdateEvent, RegisterAgentRequest messages

---

Prepared by the inventor for evaluation by patent counsel.
This document is a technical disclosure, not a patent application.